03: Foundations

ISA 401: Business Intelligence & Data Visualization

2026-08-31

Slides | Class code

The recording is on Canvas, available to ISA 401 instructors. Timestamps before each paragraph are hh:mm:ss into the recording; drag the player there to hear the passage.

Timeline

Start	End	Min	Segment
00:00:00	00:02:39	2.7	Opening: Wednesday recap, Zoom and repository housekeeping
00:02:39	00:05:49	3.2	Kahoot setup, PIN, joining
00:05:49	00:24:14	18.4	Week 01 Kahoot review, 6 questions, each one re-explained
00:24:14	00:26:59	2.8	Agenda for the day, Menti poll on prior R experience
00:26:59	00:32:49	5.8	New project, R Markdown file, `markdowns` folder, GitHub Desktop check
00:32:49	00:40:37	7.8	Project Options and Global Options, R Markdown cheat sheet
00:40:37	00:46:49	6.2	YAML header built option by option

Start	End	Min	Segment
00:46:49	00:52:40	5.8	Student screen shares, YAML errors
00:52:40	00:59:05	6.4	First code chunk, `.gitignore`, commit and push
00:59:05	01:04:22	5.3	Assignment operators, object not found, R as a vector language
01:04:22	01:10:45	6.4	Types in the class RMD: `typeof()`, the integer `L`, the native pipe, `results='hold'`
01:10:45	01:18:02	7.3	Four types on the slides, then the timed type self-assessment
01:18:02	01:20:48	2.8	Closing: commit and push, undo versus revert, what comes next

The segments sum to about 80.8 minutes, which covers the 1:20:48 recording with no unaccounted gap. Against the plan, the Kahoot, which the deck's notes place right after the setup timer so latecomers can settle, instead absorbed roughly a third of the session, Part 1 expanded into a YAML walkthrough and two screen-share repairs that the deck does not allow for, and everything in the Recap section fell off the end.

Where students struggled

Fadel re-explained every Kahoot question, not only the ones that went badly, so each entry below says whether the recording gives any sign that the room actually missed it.

Kahoot question 3: how the county map encodes counties

00:12:52 The question showed the county-level map and asked how the counties are encoded. Fadel's response to the scoreboard was to say he was going to smile, and then to go back past the question itself to two terms from Class 01.

00:13:13 The first term he re-stated was unit of analysis, which is the question of what one row of data means. In the crash dataset the unit of analysis was not a crash, it was a person involved in a crash. The second was data encoding, which he glossed as a fancy way of asking how a

variable in a file gets represented in a picture. He had already covered this in Class 01 and again on Wednesday, and said so. On this map the counties are drawn: the x and y position traces the border, and Butler County sits where it does because it is in southwestern Ohio, next to Montgomery and Warren.

00:14:35 Only then did he take the miss itself. Color is not the encoding for counties because color is not being used to tell Butler from Warren. The New York Times used color for the case count per capita, light orange for few cases and dark red or maroon for many. Nobody is saying Warren is orange and Cuyahoga green as a way of telling them apart. Fadel checked twice that this had landed, once by asking whether the room would get it right on an exam, and noted that the recording is there for it.

Kahoot question 4: where a committed file lives

00:15:43 The question was: you edit a file, save it, and commit it, so where does that file live now? Enough of the room answered that it is now on every computer where they are signed into GitHub Desktop that Fadel used the result as an argument for the class asking him more questions.

00:16:21 The cause is treating Git and GitHub as one thing. His re-explanation separated the operations. Git operations happen on the local computer. Ticking the checkbox in GitHub Desktop marks a file to be tracked, and clicking commit takes a snapshot of that file as of 8:46:59 a.m. on August 31st, which he called the equivalent of naming a version in a Google Doc history.

00:17:13 Committing puts nothing on the server; push does that. Having the file on a second computer needs two things, a push and then a pull of the latest version of the repository onto that other machine. Pushing means it is on the cloud server at github.com, nothing more, and every other computer needs its own pull.

Kahoot question 5: which files Git can diff

00:19:21 Nothing in what Fadel says marks this one as a miss, but the re-explanation is worth having because it is where the binary-versus-flat distinction got settled. Wednesday's slide split files into two colors, binary and not binary; Fadel recalled the binary one as grey, thought the other was blue, and said he did not really remember. For binary files, a Word document, an image, a PDF, Git can say the version changed but not line by line what happened. For a Markdown file, a text file, a CSV rather than an Excel file, an R file, a Python file, anything flat, Git can say exactly what changed, as the class saw on Wednesday when a line was added to the skills list in a Markdown document.

Kahoot question 6: undo versus revert

00:21:34 Fadel handled this one by asking the room to take the question apart before answering. Pushing a commit is two things, taking a snapshot of the version containing the mistake and sending that snapshot up to GitHub, so the mistake is already on the cloud. Keeping the history honest means adding a version that is the state of things before the mistake, leaving the original, the mistake and the correction all visible. The term is reverting.

00:22:47 He then walked the distractors. There is no real undo in GitHub Desktop once a commit has been pushed. Deleting the repository and starting over, which nobody picked, loses the

history. Pulling the latest repository from github.com just brings the mistake down to another machine, because the mistake is what is up there.

Project Options was greyed out

00:34:34 Someone in the room could not open Project Options at all; the menu entry was greyed out. Fadel guessed the cause immediately, which is that they had not created a project yet, and the entry is only live inside one. The fix he gave from the podium was New Project, then Existing Directory, then browse to the folder they already have, then Create Project, after which Project Options becomes available.

The Code button did not appear

00:46:49 A question from the room about the two code options led straight into the session's main error. Fadel's answer was that both `code_folding` and `code_download` put a Code button on the knitted page, `code_folding` adding show-all and hide-all and `code_download` adding the link to download the `.Rmd`, and that if the options are added and nothing appears, the problem is almost certainly the spacing. He asked anyone in that situation to say so, and someone did.

YAML indentation, on two screen shares

00:47:57 Two screen shares came up over Zoom, and in both cases the failure was in the YAML block rather than in any R. The common error in the first case is keys under `output:` that are not lined up with each other, which makes knit fail. Fadel's fix was not to count spaces but to rebuild the block from `output:` downwards, pressing Enter after each key and letting RStudio place the line below it. He said out loud while doing it that he had thought those lines needed to be indented and that they would see, and once the indentation was consistent the knit succeeded. He drew the general lesson afterwards, which is to type the key and its colon, press Enter, and let RStudio tab it rather than counting spaces.

00:51:24 The second share had a different cause. Knit produced no error message at all, and the thing to check in that case is whether part of the YAML header has gone missing rather than being misaligned. That one was resolved on the call.

Editorially: if a successor is debugging the same block, the check that separates the two cases is whether every key at the same level starts in the same column and whether a key with children ends in a colon.

crashdate and age in the type self-assessment

01:15:32 The two columns that needed the most work were the two the deck's own speaker notes predict. For `crashdate`, Fadel put a question back to the room about which function they use to read a CSV in R, and a student answered `read.csv`. He accepted that, then introduced `readr::read_csv()`, which guesses types better and recognizes this column as a date-time, which makes it a double, numeric underneath.

01:16:13 For `age` the trap is that the column should be numeric but contains some non-numeric values, and that makes the whole thing character. He turned this into the rule to carry out of the session, and noted it is not only an R rule: a vector holds one type, and mixing types generalizes

to the most general type that can hold everything. Integers and doubles together, `401L` and `44.7`, generalize to double. Numbers and text together generalize to character.

01:17:14 Working the rest of the table, `instanceid` looks like words so it is character, `dayofweek` is text so it is character, `latitude_x` has a decimal point so it is a double, and `injuries` is text. The Answer tab on the slide carries the code that produces the six answers together with the two gotchas written out. Fadel invited notes from the discussion into the editable box on that slide.

Questions from the room

00:29:10 A student asked whether someone using R in other classes should set up a separate project directory for each of them. Fadel said yes, and called it a good recommendation: a project per class. His reason was that somewhere between 10 and 20 percent of the trouble people have reading files in R is a wrong file path, and if everyone shares the same folder structure the code he posts online runs as-is, provided the person running it is inside the project and has downloaded any data file it needs.

00:30:00 Fadel then put a question back to the room, R scripts or R Markdown, and said he would follow the recommendation. The room said Markdown, and that is what the course uses from here.

00:46:49 A student asked what `code_folding` and `code_download` each do, given that both put a Code button on the page. The answer, and the spacing failure it uncovered, is in the section above.

01:15:32 Asked of the room during the self-assessment: which function do you use to read a CSV in R? Someone answered `read.csv`, which Fadel used as the way into `readr::read_csv()`.

Demo notes and gotchas

Opening housekeeping

00:00:03 Fadel opened by saying he had filled in the notes on the Week 01 slides since last time, so the decks now carry in writing what was only said aloud. His one-sentence summary of Wednesday was the motivation for R: it is a language you can read, and that lets an analysis carry from one dataset to another far faster than in a spreadsheet, at least for him.

00:01:20 The housekeeping worth repeating: join the Zoom from the Week 0 module on Canvas so anyone who hits an error can share a screen, and work from the machine that already has the repository, because files get added to it today. Fadel has RStudio pinned to his taskbar, and it is also found by typing RStudio in the Windows search bar. Assignment 0 covered installing it on personal laptops.

Running the Kahoot

00:02:39 Fadel described the format as about five of these, give or take, as a check on how the understanding is going, with six questions today evenly split between Class 01 and Class 02.

00:03:47 Players join at kahoot.it with a six-digit code, on a phone or in a browser. Two practical points he made from the podium: ask for first and last name so the bonus can be matched to a person, since duplicate first names make more work for him at grading time, and expect his own

name on the scoreboard because he had been testing the game. He told the room twice that he does not count.

00:23:39 The winner gets the 0.15 bonus on Assignment 02.

The questions actually run in the room were not the deck's bank in the deck's order. Five of the six match, but the deck's fifth question, on which visual encoding carried each day's count in the crashes-by-weekday chart, was replaced by a question on how a county-level map encodes counties, and the commit and diff questions came after it rather than before. A successor working from the deck's speaker notes should check the live Kahoot before assuming they match.

00:05:49 The content he re-taught on the way through: business intelligence lives in pre-analytics and descriptive, the Job Scout example from Class 01 being both halves at once, with scraping and organizing job ads into a database as the pre-analytics part and the interactive charts on top as the descriptive part. The data management piece is the work from 345; what this class adds is pulling data from sources outside the organization.

00:07:24 Descriptive analytics splits into statistical summaries and visualizations people can understand. Predictive is typically machine learning. Prescriptive got two buckets, experimental design of the ISA 335 kind, with Amazon moving its buy button and A/B testing whether more people buy, and the optimization side from ISA 321, where a good six-month prediction of stock performance still does not mean putting all $10,000 into one of them.

00:08:55 He closed that question on dashboarding: insight from a dashboard in Tableau or Power BI, either retrospectively, as in how did we do last quarter, or in real time, as in what is the Ford plant up in Michigan producing right now.

00:10:13 The three shapes of data came out of the JSON question, which went better. The wall of text describing a job is unstructured, because text and images are not structured by design. The skills coming back from the job-ads API arrive as JSON, more organized than free text but still needing a decision before they reach a table: all skills compressed into one cell separated by commas or semicolons, or a long table with a job ID and a skill, so a job with five skills is five rows and one with fifteen skills is fifteen. Having to make that decision is what semi-structured means. Structured data is the 345 material, anything that already sits in a data table.

00:18:32 A stance he stated in the middle of the review and repeated later: this is not a sprint, and if the room needs something explained in more detail it gets explained in more detail.

The pacing poll

00:26:00 Before the demo he ran a Mentimeter poll to set the pace, asking whether students had used R before, joinable at menti.com with a code or by scanning the QR code on the slide. The revealed board showed 28 of 28 responded: 8 for "Yes, comfortable with it", 20 for "Yes, but I do not trust my skills", and none for "No". His read to the room was that on a positive note everybody who voted had used R before, and that the class would take it one step at a time from there.

From repository to project

00:27:30 The rationale he gave for working exclusively in a project environment is that if everyone has the folders set up the same way, his code runs on their computer without editing. On the projector: File, New Project, Existing Directory, because the folder already exists from last class. His own is the `isa401a` folder on the M drive; the students' should be `business_intelligence`. Browse to it, then Create Project.

00:28:32 What the project buys is that every R session started in it begins in that folder, so paths are relative to the project root rather than absolute. He said he would come back to what that means as the semester goes on.

00:30:14 The file was created through File, New File, R Markdown, with the title `03 - R Basics` and his own name in the author field. Successors should note a mismatch with the deck: the demo slide names the file `class03.Rmd`, while the class code repository holds `markdowns/03_r_basics.Rmd`.

00:30:48 With the file open he toured the panes: the source file top left, the console below it where output appears and which he called a throwaway place to try things that should not end up in the record, the Environment pane where objects like a data frame show up, and the Files pane. The project name in the corner read `isa401a`.

00:31:27 He then made a `markdowns` folder inside the project and saved the file into it.

00:32:12 Switching to GitHub Desktop to show where that landed, the repository showed the `.Rproj` file and the new R Markdown with everything in it green, meaning none of it existed before, plus a `.Rproj.user` folder dealt with later.

Settings, and where they actually live

00:32:49 This is the part of the demo most likely to trip a successor. Fadel did the setup in Tools, Project Options, noting that the slide shows the same settings in Global Options and that off the top of his head he did not know which settings live in one and which in the other, only that they share a lot. Doing it per project is useful if another class has a faculty member with different preferences.

00:33:12 In Project Options, General, all three Workspace settings go to no: restore .RData into workspace at startup, save workspace to .RData on exit, and always save history. His reason is that a `DF` left in the environment from a different session and a different dataset makes it easy to run an analysis on the wrong data without noticing.

00:33:58 In Project Options, Code Editing, tick use native pipe operator. On Windows `Ctrl + Shift + M` then types the pipe, M as in Markdown, which is how he remembers it. He glossed the pipe as a fancy way of saying the output from the first function becomes the first input to the next.

00:36:01 Appearance is optional and he flagged it as optional twice. He uses the Tomorrow Night Bright editor theme, and the only advantage he claimed for matching it is that the colors on a student screen then match the colors on the projector. The claim that a dark theme is easier on the eye he offered as theory rather than fact.

00:36:53 One pane in Project Options he passed over without settling: he said it is actually different from Global Options, that it is not what he is used to, that he would leave it as yes or no up to the room, and that the class will not be using Python. The pane on screen at that moment is the R Markdown pane, with the Python entry visible below it in the sidebar.

00:37:09 Then Tools, Global Options for what is not available per project. Editor font size is personal and he likes it bigger. Under Code, Display, tick highlight R function calls and use rainbow parentheses. The first gives a function name its own color, distinct from the package name. The second gives each nesting level of parentheses its own color, so it is visible at a glance whether a parenthesis that was opened has been closed. Neither changes how the code runs. The effect was visible the moment he clicked Apply.

00:38:51 In the R Markdown pane of Global Options he set the equation and image previews dropdown to Never, hedging that he tends to do this "if I'm not mistaken". He then went looking for the setting that controls where chunk output appears rather than inline, could not find it in that pane, checked Advanced, and moved on with "when we run it, I'll figure it out". What he settled on later, at 01:06:37, was show output preview in, offering Window and Viewer Pane.

00:39:43 Help, Cheat Sheets, R Markdown Cheat Sheet downloads a PDF. He noted there are two R Markdown entries in that menu and could not remember which was which. The one that opened lists the YAML options and shows what each format looks like, and he confirmed on screen that it is still two pages.

Building the YAML live

00:40:37 This walkthrough is not in the deck, which simply tells students to delete everything below the setup chunk. It took roughly six minutes. Fadel defined the block between the two rows of three dashes, lines 1 to 6 of a new file, as the YAML, standing for Yet Another Markup Language, and hedged that the one-line gloss may not be fully accurate but is good enough as a mental model: this is where the metadata goes and where the look of the file is specified. Out of the box, Knit produces an HTML file next to the `.Rmd` in `markdowns/`.

00:41:31 The habit he built the rest of the demo on: YAML is whitespace-sensitive in a way R is not, so rather than count spaces, type the key and its colon first, then press Enter and let RStudio indent the next line. He wrote `html_document:` with the colon in place before pressing Enter specifically to show the automatic indent.

00:42:14 From there each option went in one at a time with a knit after each. `toc: TRUE` put a table of contents at the top, which on the template file listed R Markdown and Including Plots. He arranged the windows side by side to show source and output together.

00:42:52 `toc_float: TRUE` moved the contents to the left and made it follow the scroll, which is what long documents need. At this point he also ticked Knit on Save so that saving re-knits, which is what made the side-by-side comparison work for the rest of the hour.

00:43:26 `number_sections: TRUE` numbered the headings, turning the two template sections into 1 and 2.

00:44:18 `code_folding: show` works only in HTML documents and does two things: a Code button at the top right of the knitted page with show-all and hide-all, and a show/hide toggle per chunk.

00:44:48 `code_download: TRUE` is the one he uses so only one file has to be handed out. With it set, someone holding only the HTML can download the `.Rmd` from the page.

00:45:48 He pointed back at the cheat sheet, whose YAML section documents most of these along with what each format looks like.

00:46:13 Last step before any R: everything from the `R Markdown` heading down is template boilerplate, so he deleted it and knitted again to an empty page. He said he expects to be quicker about this in future classes and showed it today because it is an option worth knowing.

00:48:18 One request that came out of the screen shares and is worth making before the demo rather than during it: anyone joining the Zoom from their own computer should use the Zoom desktop app rather than Zoom in a browser tab, because the app lets Fadel ask for remote control and fix things directly instead of describing keystrokes.

The first chunk

00:53:33 His recommendation is never to type the three backticks by hand, but to use the insert-chunk button in the editor toolbar and pick R.

00:53:55 Reading a chunk header left to right: the programming language first, and R Markdown can run several in one document, with Python needing the reticulate package. Then the chunk name, which he recommends making descriptive and which cannot contain spaces, so `cincy crashes` has to become `cincy_crashes`. Then the chunk options.

00:54:24 Two ways to get output, and the order of operations is the only difference. Knit always saves the file first. Knit on Save is the other way round. Either way the knitted document holds code and output together.

00:58:10 Two ways to run code in a chunk: `Ctrl + Enter` for the current line, or the whole chunk at once. He writes one line at a time and runs it, to be sure it works before moving on.

.gitignore, commit, push

00:54:50 The `.Rproj.user` folder RStudio creates is not something anyone wants in a repository. In GitHub Desktop, right-click one of those changed files, choose to ignore the folder, and pick the top-level folder. He was careful to say that skipping this is not an error, just that those files are of no interest to anyone.

00:55:47 Ignoring the folder created a `.gitignore` with one line in it. On the projector this took the changes list from 19 files down to 4: the new `.gitignore`, the `.Rproj` file, which he said one might want to keep, and the `.Rmd` and `.html` in `markdowns/`.

00:56:15 The demonstration worth keeping. He typed a commit summary, misspelled it, retyped it, and committed. Then, before pushing, he opened his repository page on github.com to show that nothing new was there and the last commit was five days ago. Only after Push origin and

a refresh did the `markdowns` folder appear. That sequence is what makes the Kahoot question on commit versus push concrete, and it is worth doing in that order rather than pushing first.

00:57:26 He also spent a moment looking for a way to suppress an inline preview element in the editor and did not find it, saying there used to be a way and maybe in this version of RStudio there is not.

00:58:35 The ritual he set for every class from here: open the project, work in an R Markdown, and before leaving, knit, commit and push.

Later demo stalls

01:04:48 The `typeof()` demonstration stalled on a naming fumble, and the recovery is the useful part. Fadel could not remember whether the object was `crashes` or `crashes_per_day` or `crashes_by_day`, went back to the earlier chunk to check, and copied the name rather than retyping it.

01:06:30 The knit appeared to succeed but the output did not show, and he said he did not know why, wondering aloud whether he had suppressed something he should not have. This is what sent him back to Tools, Global Options, R Markdown, where he found show output preview in, with Window and Viewer Pane as the two choices, and set it so the preview appeared where he wanted.

01:07:43 Looking at the held output surfaced a real mistake in the class RMD: the chunk was printing `crashes_per_day` where it should have printed `crashes_by_day`. He corrected it on screen, and the knitted chunk then showed the two things it should, a single value and the named vector.

Closing the Git loop

01:18:02 With two minutes left he chose not to push into the next topic and closed the Git loop instead. The changed files showed orange rather than green because they already existed: a block of changed lines in the HTML starting around line 1600, and in the `.Rmd` the chunk gaining `results='hold'` plus a few added lines. Commit summary `end of class03`, then push.

01:19:17 The last Git point was made with the commit sitting locally and not yet pushed, and he corrected himself mid-sentence to make the condition exact: not that it was uncommitted, but that it was committed and not pushed. In that state, right-clicking the commit offers undo commit, and undo does not delete the work, it puts the changes back so you can add whatever you forgot. He used it to add a line about the slide deck not being finished, then committed again and pushed. Once a commit has been pushed that option is gone and revert is the route, which is the Kahoot question seen from the other side.

Code as taught

All of this was built in front of the room rather than opened from a finished file, but not all of it was typed. The YAML header was assembled option by option over about six minutes, with a knit after each addition. The setup chunk came with the R Markdown template and was left alone. The two lines of the first chunk were copied off the slide after a false start, and the seven

integer values in the second chunk were copied from the chunk above rather than retyped. The two blocks in the last subsection stayed on the slides and were never typed into the class RMD.

The header, built up option by option and knitted after each addition:

```
#| eval: false
---
title: "03 - R Basics"
author: "Fadel M Megahed"
date: '`r Sys.Date()`'
output:
  html_document:
    toc: TRUE
    toc_float: TRUE
    number_sections: TRUE
    code_folding: show
    code_download: TRUE
---
```

Line 8

A table of contents at the top of the document.

Line 9

Moves it to the left margin and lets it float as you scroll.

Line 10

Numbers the sections.

Line 11

HTML only. Adds a Code button with show-all and hide-all, plus a toggle per chunk.

Line 12

Puts a download link for the `.Rmd` in the HTML, so one file is enough to hand out.

The template's setup chunk, which arrived with the new file and was not discussed:

```
#| eval: false
knitr::opts_chunk$set(echo = TRUE)
```

Under the heading `# Crashes Per Day` and the subheading `## Putting the Data in RMarkdown`, the chunk named `cincy_crashes`. It grew through the session: when first written it held only the two `crashes_per_day` lines, the vector came next, and the `results='hold'` option went on during Part 2 rather than when the chunk was created.

```
#| eval: false
crashes_per_day = 30125 / 365
```

```
crashes_per_day

crashes_by_day = c(2020, 2241, 2327, 2285,
                   2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")
crashes_by_day
```

Line 2

30,125 is the row count of the crash file, so this is crashes per day for 2025.

Line 3

Typing the name on its own line prints it.

01:01:38 The teaching around that vector was that R is a vector language: a function run on a whole vector works, element by element, with no loop written anywhere. He introduced the seven numbers as the counts of crashes on each day of the week in 2025; the deck's notes identify them as the seven bars from Wednesday's chart. `c()` puts them into a vector of seven elements, and Fadel said he thinks of it as concatenating while noting he does not necessarily know whether that is technically what is happening. `names()` attaches a name to each element.

01:07:04 The `results='hold'` option, which he uses in the class notes, holds the outputs of a chunk and shows them together at the end instead of interleaving each result with the code that produced it.

Under the heading `## Type of `crashes_per_day``, written with backticks so the name renders as code when knitted, the second chunk. It is unnamed in the class RMD.

```
#| eval: false
typeof(crashes_by_day)

crashes_by_day_int = c(2020L, 2241L, 2327L, 2285L,
                   2380L, 1866L, 1664L)
names(crashes_by_day_int) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")
crashes_by_day_int |> typeof()
```

Line 2

In the console this returned `"double"`.

Line 4

The same seven numbers, copied from the chunk above rather than retyped, with an uppercase `L` after each one to make them integers. Fadel said out loud not to type these by hand.

Line 6

Copying the block over left this line still naming `crashes_by_day`; Fadel spotted it and changed it to `crashes_by_day_int`, adding that it did not really matter here, because the only thing being checked was the type.

Line 8

The native pipe: take the thing on the left and make it the first input to the function on the right. `Ctrl + Shift + M` types the pipe. This returned `"integer"`.

01:09:31 He said the pipe is unnecessary here and that he was showing it only to show it. The substance is the contrast between the two chunks: the first vector had no decimal points anywhere and R still stored it as a double, because a double is what allows decimals, so making something explicitly an integer means telling R.

Shown on the slides, not in the class RMD

00:59:05 Fadel called R a weird programming language in one specific way, that you can assign a value to an object in three different ways.

```
#| eval: false
crashes_per_day = 30125 / 365
crashes_per_day <- 30125 / 365
30125 / 365 -> crashes_per_day
```

Line 2

`=`, which he assumed is what students use in their other classes, and what this course uses.

Line 3

`<-`, literally an arrow: take this value and assign it back into the name. This is what comes back from ChatGPT or from code found online, and it was the default way people wrote R, he guessed, ten or fifteen years ago.

Line 4

`->`, pointing the other way. Very uncommon, but he does use it at the end of a series of piped steps, where saying the output goes into this object reads well.

01:00:13 All three do the same thing. He said he did not know whether other programming languages offer this; the point was only that R has three and that this is unusual. He then moved straight to R's errors, which he called awful and not very descriptive.

01:00:41 The most common error is object not found, and he simulated it live rather than showing the slide: he removed `crashes_per_day` from the environment and pressed `Ctrl + Enter` on the line that prints it, so R reported that it did not know where the object was.

01:01:11 His ranking of the three causes: an uppercase letter where there should not be one, some other typo in the name, or simply never having run the line. The check he gave is to compare the name as typed against the Environment pane, and if it is not there, that is the reason.

The vector operations, which stayed on the slide:

```
#| eval: false
crashes_by_day = c(2020, 2241, 2327, 2285,
                   2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")
crashes_by_day
crashes_by_day > 2000
sum(crashes_by_day > 2000)
mean(crashes_by_day)
```

Line 2

`c()` puts the numbers together into a vector of seven elements.

Line 4

`names()` attaches a name to each element, making it a named vector. Fadel read the printed result as Mon at 2020, Tue at 2241, and so on.

Line 7

A greater-than sign in R is a logical operation: it asks, for each element, true or false. One comparison, seven answers. On the slide, Saturday and Sunday come back `FALSE` and the other five `TRUE`.

Line 8

`sum()` of `TRUE`s and `FALSE`s counts the `TRUE`s, because `TRUE` is 1 and `FALSE` is 0. The slide shows 5.

Line 9

`mean()` works on the vector the same way, and the slide shows 2111.857.

01:03:36 His contrast was with Python, where the default way to do this would be to go through the list, typically with a list comprehension. In R you operate on the vector itself, with no `for` loop anywhere.

01:04:22 R has six data types by default, four of them common enough for this class. Logical is `TRUE` or `FALSE`. Integer is a whole number, and making something an integer in R means giving it an uppercase `L`.

01:10:45 Back on the slides for the visual view of the four types. Factor is the one students have met in regression, where days of the week as a predictor get made into a factor. A factor is another way of saying categorical variable, and it differs from a character variable in being limited to specific values: if the days of the week are the levels, there cannot be a day of the week outside them. His analogy was a select filter in a dashboard, which limits people's answers to a fixed list.

01:12:07 Integer and double are the numeric types. He misspoke here and corrected himself in the same breath: double allows decimal points, integer does not. To check the type of something, `typeof()`, which he said takes a single value or a whole vector.

The type self-assessment

01:12:33 Two minutes on the countdown to connect each of six columns from the Cincinnati crash file to its type: `instanceid`, `crashdate`, `dayofweek`, `age`, `latitude_x` and `injuries`, with seven sample rows on screen showing what the values look like.

01:13:03 He gave one caveat before starting the timer, and it is worth repeating verbatim when running this again. The note at the bottom of the slide says the data was read with `read_csv()`, and he said explicitly that some of these columns could be understood differently depending on how the file was read: a column full of dashes and colons might well come back as character, and here it does not.

01:17:43 The Answer tab on the slide carries the code producing the six answers together with the two gotchas written out, that `crashdate` is a double because readr stores a date-time as a number with a label on top, and that `age` is character because of the letter codes in it. Cleaning `age` is a Class 07 job.

For next time

00:24:14 Fadel's own remark on the overrun: the review took about 25 minutes and he did not intend it to, but his view is that if reviewing what has been covered is what it takes for everyone to be on the same page going forward, that is a win. His stated plan for the remaining time was RStudio as the interface to R, a little syntax, and a lot of indexing, which he called really basic, really fundamental, and the source of about half the errors students will get. Indexing was not reached.

00:24:47 On what he expects to skip: custom functions and conditional statements were both offered as nice-to-haves if there was time, and he said he does not think he will talk about if-else statements at all in other sections. `for` loops he committed to, maybe not next week, the week after.

00:46:29 On the YAML walkthrough, he said he will probably be quicker about it in future classes and showed it in full today because he wanted to highlight that it is an option.

01:19:45 On the schedule: the next class continues with R, data frames and subsetting. The two assignments were pushed by a day, and he told the room they can be done now because nothing in them depends on where the class stopped and there were no R questions in them. The first fifteen minutes of the next class go to a former student now at Deloitte, talking about recruiting.

01:20:29 The takeaway he asked the room to leave with: questions in class are always welcome, so stop him and ask; and anyone chatting on Zoom should use the app rather than the browser.

Suggestion: budget the Week 01 review at 25 minutes on the plan, or cut it to four questions. Six questions with a full re-explanation of each miss took 18 minutes of question time, and indexing, named at the start of Part 1 as the day's main topic, fell off the end.

Suggestion: reconcile the deck with the demo before teaching it. The “Set Up RStudio Once” panels are written against Global Options while the live setup ran in Project Options, and Fadel had to say on the projector that he did not know which settings live where. The deck also names the file `class03.Rmd` while the class code repository holds `markdowns/03_r_basics.Rmd`.

Suggestion: decide what to do about the YAML header. Building it option by option took about six minutes and is not in the deck, which simply tells students to delete everything below the setup chunk. Either give it a slide of its own or hand the header out and knit once.

Suggestion: move the colon-then-Enter indent habit in front of the typing rather than after it. Both screen shares failed in the YAML block, and that habit is the fix Fadel reached for in both cases.

Suggestion: sync the live Kahoot with the deck’s question bank. Five of the six matched; the deck’s chart-encoding question was replaced in the room by a county-map encoding question, and the order differed.

Suggestion: assume the Recap section will not be reached at this pace. If the assignments card or the closing Menti matter, put them before Part 2 or move them into the following class.

Open: what the phrase at 00:00:31 was, where the transcript has “why we’re using RMSpot”; provisionally written as R being a readable language.

Open: whether “about 5 of these, followed by 5 of these” at 00:02:39 names a second tool in its second half, or is just a repetition.

Open: the two course numbers at 00:07:43, heard as “IA35” and “ISA 3.21” and written as ISA 335 and ISA 321.

Open: the two course numbers at 00:12:01, heard as “391” and “225”.

Open: the name of the job-ads API at 00:10:43, transcribed as “Active.cv”.

Open: the garbled sentence at 01:20:01 about the guest speaker and about what follows the guest slot.

Open: which Project Options control Fadel was leaving to preference at 00:36:53 when he said it differs from Global Options and that the class will not be using Python.

Open: whether the setting he could not find at 00:38:51 is the show output preview in setting he reached at 01:06:37, or something else in that pane.

Open: what he was trying to suppress in the R Markdown editor at 00:57:26, which he could not find in this version of RStudio.

Open: which color meant binary and which meant flat files on Wednesday’s slide; at 00:19:21 Fadel said grey for binary and was unsure of the other.

Open: when indexing, custom functions, conditional statements and `for` loops actually land, now that none of them were reached.

Materials

- Slides
- Class code
- The recording is on Canvas, available to ISA 401 instructors.